III B.TECH – I SEMESTER

Artificial Intelligence Lab

Core JNTUA R23 · B.Tech CSE

	ARTIFICIAL INTELLIGENCE LAB	L	T	P	C
		0	0	3	1.5

Course Objectives:

- To teach the methods of implementing algorithms using artificial intelligence techniques
- To illustrate search algorithms

To demonstrate the building of intelligent agents

Course Outcomes:

After completion of the course, students will be able to

- Implement search algorithms
- Solve Artificial intelligence problems
- Design chatbot and virtual assistant

List of Experiments:

- Write a program to implement DFS and BFS
- Write a Program to find the solution for traveling salesman Problem
- Write a program to implement Simulated Annealing Algorithm
- Write a program to find the solution for the wumpus world problem
- Write a program to implement 8 puzzle problem
- Write a program to implement Towers of Hanoi problem
- Write a program to implement A* Algorithm
- Write a program to implement Hill Climbing Algorithm
- Build a Chatbot using AWS Lex, Pandora bots.
- Build a bot that provides all the information related to your college.
- Build a virtual assistant for Wikipedia using Wolfram Alpha and Python
- The following is a function that counts the number of times a string occurs in another string: # Count the number of times string s1 is found in string s2 defcountsubstring(s1,s2):

count = 0

for i in range(0,len(s2)-len(s1)+1): if s1 == s2[i:i+len(s1)]:

count += 1 return count

For instance, count substring ('ab','cabalaba') returns 2.

Write a recursive version of the above function. To get the rest of a string (i.e. everything but the first character).

- Higher order functions. Write a higher-order function count that counts the number of elements in a list that satisfy a given test. For instance: count (lambda x: x>2, [1, 2, 3, 4, 5]) should return 3, as there are three elements in the list larger than 2. Solve this task without using any existing higher order function.
- Brute force solution to the Knapsack problem. Write a function that allows you to generate random problem instances for the knapsack program. This function should generate a list of items containing N items that each have a unique name, a random size in the range 1 ..... 5 and a random value in the range 1..... 10.

Next, you should perform performance measurements to see how long the given knapsack solver take to solve different problem sizes. You should perform at least 10 runs with different randomly generated problem instances for the problem sizes 10,12,14,16,18,20 and 22. Use a backpack size of 2:5 x N for each value problem size N. Please note that the method used to generate random numbers can also affect performance, since different distributions of values can make the initial conditions of the problem slightly more or less demanding.

How much longer time does it take to run this program when we increase the number of items? Does the backpack size affect the answer?

Try running the above tests again with a backpack size of 1 x N and with 4:0 x N.

- Assume that you are organising a party for N people and have been given a list L of people who, for social reasons, should not sit at the same table. Furthermore, assume that you have C tables (that are infinitely large).

Write a function layout (N,C,L) that can give a table placement (i.e. a number from 0 : : :C -1) for each guest such that there will be no social mishaps.

For simplicity we assume that you have a unique number 0 ……N-1 for each guest and that the list of restrictions is of the form [(X, Y) …] denoting guests X, Y that are not allowed to sit together. Answer with a dictionary mapping each guest into a table assignment, if there are no possible layouts of the guests you should answer False.

References:

- David Poole, Alan Mack worth, Randy Goebel," Computational Intelligence: a logical approach", Oxford University Press, 2004.
- G. Luger, "Artificial Intelligence: Structures and Strategies for complex problem solving", Fourth Edition, Pearson Education, 2002.
- J. Nilsson, "Artificial Intelligence: A new Synthesis", Elsevier Publishers, 1998.
- Artificial Neural Networks, B. Yagna Narayana, PHI
- Artificial Intelligence, 2nd Edition, E.Rich and K.Knight, TMH.
- Artificial Intelligence and Expert Systems, Patterson, PHI.

Online Learning Resources/Virtual Labs:

https://www.tensorflow.org/https://pytorch.org/ https://github.com/pytorch https://keras.io/ https://github.com/kerasteam http://deeplearning.net/software/theano/ https://github.com/Theano/Theanohttps://caffe2.ai/ https://github.com/caffe2

https://deeplearning4j.org/Scikit-learn:https://scikit-learn.org/stable/ https://github.com/scikit-learn/scikit-learn

https://www.deeplearning.ai/

https://opencv.org/

https://github.com/qqwweee/keras-yolo3 https://www.pyimagesearch.com/2018/11/12/yolo-object-detection-with-opencv/

https://developer.nvidia.com/cuda-math-library http://vlabs.iitb.ac.in/vlabs-dev/labs/machine_learning/labs/index.php